**Resolution – Ex 1**

- John likes all kinds of food
- Apples are food.
- Chicken is food.
- Anything anyone eats and isn't killed by is food.
- Bill eats peanuts and is still alive.
- Sue eats everything Bill eats
- Query: John likes peanuts

**Resolution – Ex 2**

- Marcus was a man.
- Marcus was a Roman.
- All men are people.
- Caesar was a ruler.
- All Romans were either loyal to Caesar or hated him
- Everyone is loyal to someone.
- People only try to assassinate rulers they are not loyal to.
- Marcus tried to assassinate Caesar.
- Query: Did Marcus hate Caeser

**Resolution – Ex 3**

- Steve only likes easy courses.
- Science courses are hard.
- All the courses in the basketweaving department are easy.
- BK301 is a basketweaving course.
- What course would Steve like?

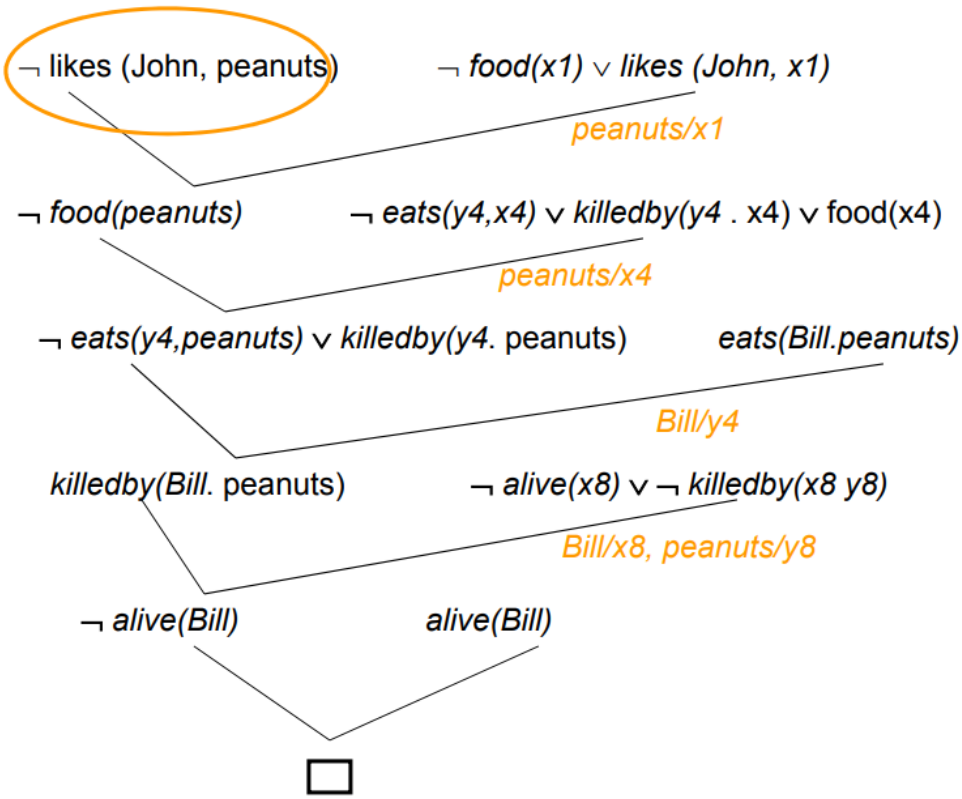
Solution

1. **John likes all kinds of food.**
   $\forall x: food(x) \rightarrow likes(john, x)$
2. **Apples are food.**
   $food(apple)$
3. **Chicken is food.**
   $food(chicken)$
4. **Anything anyone eats and isn't killed by is food.**
   $\forall x: (\exists y: eats(y, x) \wedge \neg killedby(y, x)) \rightarrow food(x)$
5. **Bill eats peanuts and is still alive.**
   A. $eats(Bill, peanuts)$ B. $alive(Bill)$
6. **Sue eats everything Bill eats.**
   $\forall x: eats(Bill, x) \rightarrow eats(Sue, x)$
7. $\forall x: \forall y: alive(x) \rightarrow \neg killedby(x,y)$

## Clause form:

1. $\neg food(x1) \vee likes(John, x1)$
2. $food(apples)$
3. $food(chicken)$
4. $\neg eats(y4,x4) \vee killedby(y4, x4) \vee food(x4)$
5. $Eats(Bill, peanuts)$
6. $Alive(Bill)$
7. $\neg eats(Bill,x7) \vee eats(Sue,x7)$
8. $\neg alive(x8) \vee \neg killedby(x8, y8)$

# Resolution proof that John likes peanuts




**1) The (relevant) clauses (CNF / clause form)**

Labelled for reference:

C1. `¬likes(John, peanuts)`
*(the negated goal we add to the KB to prove `likes(John,peanuts)` by refutation)*

C2. `¬food(x1) ∨ likes(John, x1)`
*("If x is not food then John likes x" is weird English but here the clause is what the diagram shows — treat it as a given rule)*

C3. `¬eats(y4, x4) ∨ killedby(y4, x4) ∨ food(x4)`
*(either y4 does not eat x4, or y4 is killed by x4, or x4 is food)*

C4. `eats(Bill, peanuts)`
*(Bill eats peanuts — a ground fact)*

C5. `¬alive(x8) ∨ ¬killedby(x8, y8)`
*(if someone was killedby something then they are not alive; written as a clause that says: either not alive(x) or not killedby(x,y))*

C6. `alive(Bill)`
*(Bill is alive — ground fact)*

(These match the lines in the picture; variables `x1,x4,y4,x8,y8` will get instantiated during resolution.)

**2) Resolution sequence (each step: which clauses resolve, the unifier, and the new clause)**

**Step A: resolve C1 with C2**

- Clauses:
  - C1: `¬likes(John, peanuts)`
  - C2: `¬food(x1) ∨ likes(John, x1)`
- Unifier: `x1 = peanuts` (to match `likes(John,x1)` with `likes(John,peanuts)`)
- Resolvent: eliminate `likes(John,peanuts)` and `¬likes(John,peanuts)` → get
  **R1:** `¬food(peanuts)`

**Meaning:** from assuming `¬likes(John,peanuts)` and the general rule C2, we infer `¬food(peanuts)`.

**Step B: resolve R1 (`¬food(peanuts)`) with C3**

- Clauses:
  - R1: `¬food(peanuts)`
  - C3: `¬eats(y4, x4) ∨ killedby(y4, x4) ∨ food(x4)`
- Unifier: `x4 = peanuts` (to match `food(x4)`)
- Resolvent: removing `food(peanuts)` gives
  **R2:** `¬eats(y4, peanuts) ∨ killedby(y4, peanuts)`

**Meaning:** either nobody eating peanuts, or whoever eats them is killed by them (per the given rule).

**Step C: resolve R2 with C4 (`eats(Bill,peanut`**

- Clauses:

**Step C: resolve R2 with C4 (`eats(Bill,peanuts)`)**

- Clauses:
  - R2: `¬eats(y4, peanuts) ∨ killedby(y4, peanuts)`
  - C4: `eats(Bill, peanuts)`
- Unifier: `y4 = Bill`
- Resolvent: `killedby(Bill, peanuts)` (because `¬eats(Bill,peanuts)` cancels with `eats(Bill,peanuts)`)

**Result: R3:** `killedby(Bill, peanuts)`

**Meaning:** from Bill eating peanuts and the rule, we conclude Bill is killed by peanuts.

**Step D: resolve R3 (`killedby(Bill,peanuts)`) with C5 (`¬alive(x8) ∨ ¬killedby(x8,y8)`)**

- Clauses:
  - R3: `killedby(Bill, peanuts)`
  - C5: `¬alive(x8) ∨ ¬killedby(x8, y8)`
- Unifier: `x8 = Bill`, `y8 = peanuts`
- Resolvent: `¬alive(Bill)`

(because `¬killedby(Bill,peanuts)` canceled with `killedby(Bill,peanuts)`)

**R4:** `¬alive(Bill)`

**Meaning:** whoever is killed by something is not alive — so Bill is not alive.

**Step E: resolve R4 (`¬alive(Bill)`) with C6 (`alive(Bill)`)**

**Step E: resolve R4 (`¬alive(Bill)`) with C6 (`alive(Bill)`)**

- Clauses:
  - R4: `¬alive(Bill)`
  - C6: `alive(Bill)`
- These are direct complements; resolving them yields the **empty clause** □ (contradiction).

**3) Conclusion of the refutation**

- We started by **adding the negation** of the claim we wanted to prove: `¬likes(John,peanuts)` (C1).
- Through a chain of sound resolution steps and unifications we derived a contradiction (□).
- Therefore the initial negation is inconsistent with the KB. By refutation completeness, the KB **entails** the positive goal:

$$\mathbf{KB} \models likes(John, peanuts).$$

In words: *the knowledge base logically implies that John likes peanuts.*

**4) Notes on mechanics and why this works**

- **Resolution rule**: given clauses containing a literal `L` and another clause containing `¬L'` that unify with substitution `θ` (i.e. `Lθ = L'θ`), you may infer the disjunction of the remaining literals from both clauses with `θ` applied. That's what we did at every step (we unified variables with ground terms like `peanuts` or `Bill`).
- **Unification** is the process used to find substitutions such as `x1 = peanuts`, `y4 = Bill`, etc.
- This example is a **refutation proof** (proof by contradiction): add negated conjecture, derive contradiction ⇒ original conjecture follows.
- The proof used only a small fragment of the KB. In real first-order resolution you must first transform formulas to **CNF** and usually **Skolemize** existential quantifiers; here the clauses were presented already in clause form.


- This example is a **refutation proof** (proof by contradiction): add negated conjecture, derive contradiction ⇒ original conjecture follows.
- The proof used only a small fragment of the KB. In real first-order resolution you must first transform formulas to **CNF** and usually **Skolemize** existential quantifiers; here the clauses were presented already in clause form.


**5) Intuition about the KB chain of reasoning**

Putting the natural-language chain together:

1. Suppose John does **not** like peanuts. (assume `¬likes(John,peanuts)`)
2. From the rule about `likes` (C2) we get `¬food(peanuts)`.
3. From the food/eats/killedby rule (C3), and ¬food(peanuts), we get either nobody eats peanuts or whoever eats them is killed by them.
4. Bill eats peanuts (C4), so from step 3 we infer Bill is killed by peanuts.
5. From the killedby → not alive rule (C5), Bill is not alive.
6. But KB also contained `alive(Bill)` (C6). Contradiction.
7. Therefore our initial assumption `¬likes(John,peanuts)` must be false, so `likes(John,peanuts)` must be true.

Ex 2

# Convert to First order Logic

(a) Marcus was a man.

(b) Marcus was a Roman.

(c) All men are people.

(d) Caesar was a ruler.

(e) All Romans were either loyal to Caesar or hated him (or both).

(f) Everyone is loyal to someone.

(g) People only try to assassinate rulers they are not loyal to.

(h) Marcus tried to assassinate Caesar.

(a) man(marcus)

(b) roman(marcus)

(c) $\forall X.\ man(X) \rightarrow person(X)$

(d) ruler(caesar)

(e) $\forall X.\ roman(x) \rightarrow loyal(X,caesar) \vee hate(X,caesar)$

(f) $\forall X \exists Y.\ loyal(X,Y)$

(g) $\forall X \forall Y.\ person(X) \wedge ruler(Y)\ tryassasin(X,Y) \rightarrow \neg loyal(X,Y)$

(h) tryassasin(marcus,caesar)

# Convert to Clausal Form

1. man(marcus)
2. roman(marcus)
3. $(\neg man(X), person(X))$
4. ruler(caesar)
5. $(\neg roman(X), loyal(X,caesar), hate(X,caesar))$
6. (loyal(X,f(X))
7. $(\neg person(X), \neg ruler(Y), \neg tryassasin(X,Y), \neg loyal(X,Y))$
8. tryassasin(marcus,caesar)